

TreasureDAO-8 Preliminary Audit Report (external, private)

⛑️ TreasureDAO-8 Fixes

Initial Code Freeze Commit Hash: `de3e1c3b42e493bc2ed1ba6a43248d39cd455e20`

Issues Found

Severity	Tag	Count
Critical	(C-1)	
High	(H-1)	
Medium	(M-1)	1
Low	(L-1)	2
Code Quality	(Q-1)	4
Gas Optimization	(G-1)	
Informational	(I-1)	2

[M-1] `quoteTransferRemote` should include token approvals

Topic	Usability
Impact	Medium
Likelihood	Medium

MagicBridgeArb inherits from TokenRouter that provides a function `quoteTransferRemote` to return the payment required to dispatch a message.

```
/** * @notice Returns the gas payment required to dispatch a message to the given domain's router. * @param _destination The domain of the router. * @param _recipient The address of the recipient on the destination chain. * @param _amount The amount of tokens to be sent to the remote recipient. * @dev This should be overridden for warp routes that require additional fees/approvals. * @return quotes Indicate how much of each token to approve and/or send. */ function quoteTransferRemote( uint32 _destination, bytes32 _recipient, uint256 _amount )
```

Reference: TokenRouter.sol#L184-L194

In the natspec comments on the above function it is stated:

> @dev This should be overridden for warp routes that require additional fees/approvals.

The MagicBridgeArb contract requires token approval for the amount to be locked when bridging. However, this approval requirement is not reflected in the `quoteTransferRemote` function, resulting in incorrect quotes.

Remediation to Consider

Add the token approval requirements to the quotes array returned by `quoteTransferRemote`. This should be implemented similar to the approach in HypERC20Collateral.sol#L77.

[L-1] `getBurner` returns incorrect value

Topic	Incorrect data
Impact	Low
Likelihood	Low

In MagicBridgeFacet, the `getBurner()` function should return the address of the MagicBurner contract. However, it incorrectly returns the `_magicBridge` address instead.

Remediation to Consider

Modify the function to return the `_magicBurner` address instead of `_magicBridge`.

[L-2] `onlyBurner` modifier throws incorrect errors

Topic	Errors
Impact	Low
Likelihood	Low

In MagicBridgeFacet, the onlyBurner() modifier looks like follows:

```
modifier onlyBurner() { MagicBridgeStorage.Layout storage $ =
MagicBridgeStorage.layout(); require($._magicBurner != address(0),
'Unitialized bridge'); require($._magicBurner == msg.sender, 'Caller
must be bridge'); _; }
```

Reference: MagicBridgeFacet.sol#L60-L65

As seen above, the error messages incorrectly refer to the bridge instead of the burner.

Remediation to Consider

Change the error messages to "Uninitialized burner" and "Caller must be burner".

[Q-1] Emit transaction id when withdrawing excess collateral

Topic	Events
Impact	Medium

In MagicBridgeArb's `withdrawToL1MagicBurner`, collateral is burned by initiating a transfer using the Arbitrum native bridge:

```
IL2GatewayRouter($._l2GatewayRouter).outboundTransfer($._l1Magic,
$._l1MagicBurner, amount_, ''); emit WithdrawToL1($._l1MagicBurner,
msg.sender, amount_, $._l1Magic, $._l2Magic);
```

Reference: MagicBridgeArb.sol#L117

The `outboundTransfer` method returns a unique ID that can be used to identify the L2 to L1 transaction. This identifier is particularly useful for tracking messages in the Hyperlane explorer when troubleshooting failed message deliveries.

Remediation to Consider

Emit the unique transaction ID returned by `outboundTransfer` in the `WithdrawToL1` event.

[Q-2] Move zero-address checks into `initialize`

Topic	Best Practice
Impact	Low

In MagicBridgeArb's `withdrawToL1MagicBurner` function, the following zero-address checks are performed:

```
require($._l2Magic != address(0), UninitializedL2Magic());
require($._l2GatewayRouter != address(0),
UninitializedL2GatewayRouter()); require($._l1Magic != address(0),
UninitializedL1Magic()); require($._l1MagicBurner != address(0),
UninitializedL1MagicBurner());
```

Reference: MagicBridgeArb.sol#L112-L115

Similar zero-address checks for the `_l2Magic` address are also performed in the `_transferFromSender` and `_transferTo` functions. These checks should be moved to the `initialize` function to fail early and prevent deployment of a misconfigured contract.

The same recommendation applies to MagicBridgeL1's `_transferFromSender` and `_transferTo` functions.

Remediation to Consider

Move the zero-address checks into the `initialize` function.

[Q-3] Emit an event in `setArbBridgeOutbox`

Topic	Events
Impact	Medium

MagicBurner's `setArbBridgeOutbox` function changes the contract state but doesn't emit an event.

Remediation to Consider

Consider emitting an event to enable offchain indexers to track the changes.

[Q-4] Nitpicks

Topic	Nitpicks
Impact	Low

- `supportsInterface` overrides are not needed in MagicBridge.sol, MagicBridgeArb.sol, and MagicBridgeL1.sol.
- Import of IERC20 in MagicBridge.sol is unused.
- Import of SafeERC20 in MagicBridgeL1.sol is unused.
- In MagicBridgeArb.sol#L129, the comment "*Moves user's legacy MAGIC to an escrow contract to be later burned on L1.*" is misleading, since tokens are not moved to a separate contract.

[I-1] Potential collateral shortfall in `MagicBridgeArb`

MagicBridgeArb contract implements a collateral-based locking and unlocking mechanism to facilitate bridging of MAGIC tokens on Arbitrum. Specifically:

- When MAGIC tokens are **bridged *out*** from Arbitrum to another chain, an equivalent amount is **locked** in the `MagicBridgeArb` contract.
- When MAGIC tokens are **bridged *in*** to Arbitrum, the contract **unlocks** the requested amount of MAGIC tokens to the recipient.

This approach is necessary because the Hyperlane Arbitrum bridge must interact with the legacy MAGIC token on Arbitrum, rather than minting a new token representation.

However, this approach comes with a potential risk: If more tokens are bridged into Arbitrum than out of it, the contract may not hold enough locked MAGIC tokens to satisfy the unlock requests. This would result in failed or delayed transfers for users attempting to bridge tokens to Arbitrum.

It must be ensured that the MagicBridgeArb contract always has enough collateral (MAGIC tokens) to fulfill unlock requests.

[I-2] Bridges use different types of storage patterns

The bridge contracts - MagicBridge, MagicBridgeArb, and MagicBridgeL1 - implement different storage patterns. They inherit from TokenRouter, which uses sequential storage, while also inheriting OpenZeppelin's upgradeable contracts, which use unstructured storage (EIP-7201).

While this mixed pattern isn't problematic in itself, it requires careful attention when upgrading these contracts to prevent storage collisions.